Lecture Prof. Geoffrey Hinton

Prof. Geoffrey Hinton expressed that his lecture might disappoint computer science and machine learning enthusiasts as he intended to give a genuine public lecture. He aimed to explain neural networks, language models, and his belief in their understanding. He also planned to discuss AI threats and the differences between digital and analogue neural networks, which he finds concerning.

Since the 1950s, two paradigms of intelligence have existed: the logic-inspired approach, which emphasizes reasoning through symbolic rules, and the biologically inspired approach, which prioritizes learning through neural network connections. Hinton was advised as a student to avoid focusing on learning, which was then considered a future concern.

Hinton described artificial neural networks, particularly a simple type with input and output neurons, and intermediate or hidden layers. Input neurons might represent pixel intensity in an image, while output neurons could represent object classes like 'dog' or 'cat'. Hidden neurons learn to detect relevant features for classification.

He explained the process of feature detection in neural networks using the example of identifying a bird in an image. Starting with edge detection, subsequent layers identify more complex features like beaks or circles, leading to the final output neuron that recognizes a bird based on the combination of detected features.

Neural networks learn through adjusting the weights on connections, represented by red and green dots. Hinton discussed the mutation method of weight adjustment, which is similar to evolution and involves random changes to see if performance improves. This method is inefficient for neural networks since we have a model of the network's processes.

Backpropagation is a more efficient method for adjusting weights in neural networks. It involves sending information about the desired output back through the network to compute adjustments for each weight simultaneously. This method, using calculus and the chain rule, is vastly more efficient than the mutation method.

Neural networks are used for tasks like object recognition in images. Hinton highlighted the success of his students, Ilya Sutskever and Alex Krizhevsky, in 2012, who demonstrated a neural network's ability to identify a thousand different object types with a lower error rate than conventional vision systems.

Hinton addressed the symbolic AI community's skepticism towards neural networks handling language, citing Chomsky's influence in convincing many that language is not learned. He argued against this, noting the success of neural networks in learning both syntax and semantics from data.

He discussed his 1985 work on a language model trained with backpropagation, which he considers an ancestor of current large models. This model aimed to unify two theories of meaning: the structuralist theory, which relies on word relations, and the psychological theory, which associates words with a set of features.

Hinton's model learned semantic features for each word and how these features interact to predict the next word, without relying on an explicit relational graph. This approach allowed the model to infer relationships and knowledge without storing explicit rules, similar to how large language models operate today.

Large language models, according to Hinton, are descendants of his early model but with more words, layers, and complex interactions. He refuted claims that these models are not truly intelligent, arguing that they understand by fitting a model to data through feature interactions.

Hinton addressed the "autocomplete objection," clarifying that large language models (LLMs) do not simply store word sequences but instead predict the next word through complex feature interactions. He asserted that the understanding demonstrated by LLMs is a form of modeling discrete symbol strings.

He discussed the risks associated with powerful AI, including fake media, job losses, surveillance, lethal autonomous weapons, and cybercrime. Hinton emphasized his concern about the existential threat of AI potentially wiping out humanity if misused or if it gains too much control.

Hinton speculated on the future of superintelligences, suggesting that they could be used by bad actors for manipulation and warfare. He also raised concerns about the inherent drive for control in intelligent agents, which could lead to them seeking more power to

achieve their goals more effectively.

Reflecting on the nature of digital versus analogue computation, Hinton suggested that while digital computation is energy-intensive and immortal, analogue computation could be more energy-efficient but mortal. He pondered the implications of hardware and software integration and the challenges of transferring knowledge between systems.

Hinton concluded that digital computation, despite its energy costs, might surpass biological computation in terms of efficiency and knowledge sharing. He estimated a 50% chance that AI could become smarter than humans within 20 years and expressed the need to consider how to manage more intelligent entities.

I'm going to disappoint all the people in computer science and machine learning because I'm going to give a genuine public lecture.
I'm going to try and explain what neural networks are, what language models are. Why I think they understand.
I have a whole list of those things, and at the end I'm going to talk about some threats from AI just briefly
and then I'm going to talk about the difference between digital and analogue neural networks and why that difference is, I think is so scary.
So since the 1950s, there have been two paradigms for intelligence. The logic inspired approach thinks the essence of intelligence is reasoning,
and that's done by using symbolic rules to manipulate symbolic expressions.
They used to think learning could wait. I was told when I was a student didn't work on learning. That's going to come later once we understood how to represent things.
The biologically inspired approach is very different. It thinks the essence of intelligence is learning the strengths of connections
in a neural network and reasoning can wait and don't worry about reasoning for now. That'll come later.
Once we can learn things. So now I'm going to explain what artificial neural nets are and those people who know can just be amused. A simple kind of neural that has input neurons and output neurons.
So the input neurons might represent the intensity of pixels in an image. The output neurons might represent the classes of objects in the image like dog or cat.
And then there's intermediate layers of neurons, sometimes called hidden neurons, that learn to detect features that are relevant for finding these things.

So one way to think about this, if you want to find a bird image, it would be good to start with a layer of feature detectors
that detected little bits of edge in the image, in various positions, in various orientations. And then you might have a layer of neurons
detecting combinations of edges, like two edges that meet at a fine angle, which might be a beak
or might not, or some edges forming a little circle. And then you might have a layer of neurons that detected things like a circle
and two edges meeting that looks like a beak in the right spatial relationship, which might be the head of a bird.
And finally, you might have and output neuron that says, if I find the head of a bird, a the foot of a bird, a the wing of a bird, it's probably a bird.
So that's what these things are going to learn to be. Now, the little red and green dots are the weights on the connections
and the question is who sets those weights? So here's one way to do it that's obvious.
to everybody that it'll work and it's obvious it'll take a long time. You start with random weights, then you pick one weight at random like a red dot
and you change it slightly and you see if the network works better. You have to try it on a whole bunch of different cases
to really evaluate whether it works better. And you do all that work just to see if increasing this weight
by a little bit or decreasing by a little bit improves things. If increasing it makes it worse, you decrease it and vice versa.
That's the mutation method and that's sort of how evolution works for evolution is sensible to work like that
because the process that takes you from the genotype to the phenotype is very complicated and full of random external events.
So you don't have a model of that process. But for neural nets it's crazy
because we have, because all this complication is going on in the neural net, we have a model of what's happening
and so we can use the fact that we know what happens in that forward pass instead of measuring how changing a weight would affect things,
we actually compute how changing weight would affect things. And there's something called back propagation where you send information back through the network.
The information is about the difference between what you got to what you wanted and you figure out for every weight in the network at the same time
whether you ought to decrease it a little bit or increase it a little bit to get more like what you wanted.
That's the back propagation algorithm. You do it with calculus in the cain rule, and that is more efficient than the mutation method by a factor of the number of weights in the network.
So if you've got a trillion weights in your network, it's a trillion times more efficient.
So one of the things that neural networks often use for is recognizing objects in images.
Neural networks can now take an image like the one shown and produce actually a caption for the image, as the output.
And people try with symbolic to do that for many years and didn't even get close.
It's a difficult task.

We know that the biological system does it with a hierarchy features detectors, so it makes sense to train neural networks in that.
And in 2012, two of my students Ilya Sutskever and Alex Krizhevsky
with a little bit of help from me, showed that you can make a really good neural network this way
for identifying a thousand different types of object. When you have a million training images. Before that, we didn't have enough training images and
it was obvious to Ilya who's a visionary. That if we tried
the neural nets we had then on image net they would win. And he was right. They won rather dramatically.
They got 16% errors and the best conventional could be division systems got more than 25% errors.
Then what happens was very strange in science. Normally in science, if you have two competing schools,
when you make a bit of progress, the other school says are rubbish. In this case, the gap was big enough that the very best researchers
Jitendra Malik and Andrew Zisswerman Just Andrew Zisswerman sent me email saying This is amazing and switched what he was doing and did that
and then rather annoyingly did it a bit better than us.
What about language? So obviously the symbolic AI community
who feels they should be good at language and they've said in print, some of them that
these feature hierarchies aren't going to deal with language and many linguists are very skeptical.
Chomsky managed to convince his followers that language wasn't learned. Looking back on it, that's just a completely crazy thing to say.
If you can convince people to say something is obviously false, then you've got them in your cult.
I think Chomsky did amazing things, but his time is over.
So the idea that a big neural network with no innate knowledge could actually learn both the syntax
and the semantics of language just by looking at data was regarded as completely crazy by statisticians and cognitive scientists.
I had statisticians explain to me a big model has 100 parameters. The idea of learning a million parameters is just stupid.
Well, we're doing a trillion now.
And I'm going to talk now about some work I did in 1985. That was the first language model to be trained with back propagation.
And it was really, you can think of it as the ancestor of these big models now. And I'm going to talk about it in some detail, because it's so small
and simple that you can actually understand something about how it works. And once you understand how that works, it gives you insight into what's going
on in these bigger models. So there's two very different theories of meaning, this kind of structuralist
theory, where the meaning of a word depends on how it relates to other words. That comes from Saussure and symbolic
AI really believed in that approach. So you'd have a relational graph where you have nodes for words
and arcs of relations and you kind of capture meaning like that,

and they assume you have to have some structure like that. And then there's a theory that was in psychology since the 1930s or possibly before that.
The meaning of a word is a big bunch of features. The meaning of the word dog is that it's animate
and it's a predator and so on.
But they didn't say where the features came from or exactly what the features were.
And these two thories of meanings sound completely different.
And what I want to show you is how you can unify those two theories of meaning.
And I do that in a simple model in 1985,
but it had more than a thousand weights in it.
The idea is we're going to learn a set of semantic features for each word, and we're going to learn how the features of words should interact
in order to predict the features of the next word. So it's next word prediction. Just like the current language models, when you fine tune them.
But all of the knowledge about how things go together is going to be in these feature interactions.
There's not going to be any explicit relational graph. If you want relations like that, you generate them from your features.
So it's a generative model and the knowledge is in the features that you give to symbols.
And in the way these features interact. So I took some simple relational information two family trees.
They would deliberately isomorphic morphic my Italian graduate student
always had the Italian family on top.
You can express that same information as a set of triples. So if you use the twelve relationships found there,
you can say things like Colin has Father James and Colin has Mother Victoria, from which you can infer in this nice simple
world from the 1950s where that James has wife Victoria,
and there's other things you can infer. And the question is, if I just give you some triples,
how do you get to those rules? So what is symbolic AI person will want to do
is derive rules of the form. If X hass mother Y and Y has husbands Z then X has Father Z.
And what I did was take a neural net and show that it could learn the same information.
But all in terms of these feature interactions now for very discrete
rules that are never violated like this, that might not be the best way to do it. And indeed symbolic people try doing it with other methods.
But as soon as you get rules that are a bit flaky and don't always apply, then neural nets are much better.
And so the question was, could a neural net capture the knowledge that is symbolic person would put into the rules by just doing back propagation?
So the neural net look like this: There's a symbol representing the person, a symbol representing the relationship. That symbol then via some connections went to a vector of features,
and these features were learned by the network. So the features for person one and features for the relationship.
And then those features interacted and predicted the features for the output person from which you predicted the output person you find the closest match with the last.

So what was interesting about this network was that it learned sensible things.
If you did the right regularisation, the six feature neurons. So nowadays these vectors are 300 or a thousand long. Back
then they were six long. This was done on a machine that took 12.5 microseconds to do a floating point multiplier,
which was much better than my apple two which took two and a half milliseconds to multiply.
I'm sorry, this is an old man. So it learned features
like the nationality, because if you know person one is English, you know the output is going to be English.
So nationality is a very useful feature. It learned what generation the person was.
Because if you know the relationship, if you learn for the relationship
that the answer is one generation up from the input and you know the generation of the input, you know the generation
of the output, by these feature interactions. So it learned all these the obvious features of the domain and it learned
how to make these features interact so that it could generate the output. So what had happened was had shown symbols strings
and it created features such that the interaction between those features could generate the symbol strings,
but it didn't store symbols strings, just like GPT 4. That doesn't store any sequences of words
in its long term knowledge. It turns them all into weights from which you can regenerate sequences.
But this is a particularly simple example of it where you can understand what it did.
So the large language models we have today, I think of as descendants of this tiny language model,
they have many more words as input, like a million, a million word fragments.
They use many more layers of neurons, like dozens.
They use much more complicated interactions. So they didn't just have a feature affecting another feature. They sort of match to feature vectors.
And then let one vector effect the other one a lot if it's similar, but not much of it's different. And things like that.
So it's much more complicated interactions, but it's the same general framework, the the same general idea of
let's turn simple strings into features for word fragments and interactions between these feature vectors.
That's the same in these models. It's much harder to understand what they do.
Many people, particularly people from the Chomsky School, argue
they're not really intelligent, they're just a form of glorified auto complete that uses statistical regularities to pastiche together pieces of text
that were created by people. And that's a quote from somebody.
So let's deal with the autocomplete objection. when someone says it's just auto complete.
They are actually appealing to your intuitive notion how autocomplete works. So in the old days autocomplete would work by you'd store
say, triples of words that you saw the first two. You count how often that third one occurred.
So if you see fish and, chips occurs a lot after that. But hunt occurs quite often too.
So chips is very likely and hunt's quite likely,

and although is very unlikely. You can do autocomplete like that,
and that's what people are appealing to when they say it's just autocomplete, it's a dirty trick, I think because that's not at all how LLM's predict the next word.
They turn words into features, they make these features interact, and from those feature interactions they predict the features of the next word.
And what I want to claim is that these
millions of features and billions of interactions between features that they learn, are understanding. What they're really doing
these large language models, they're fitting a model to data. It's not the kind of model statisticians thought much about until recently.
It's a weird kind of model. It's very big. It has huge numbers of parameters, but it is trying to understand
these strings of discrete symbols by features and how features interact.
So it is a model. And that's why I think these things really understanding.
One thing to remember is if you ask, well, how do we understand? Because obviously we think we understand.
Well, many of us do anyway. This is the best model we have of how we understand.
So it's not like there's this weird way of understanding that these AI systems are doing and then this how the brain does it.
The best that we have, of how the brain does it, is by assigning features to words and having features, interactions.
And originally this little language model was designed as a model of how people do it.
Okay, so I'm making the very strong claim these things really do understand.
Now, another argument people use is that, well, people GPT4 just hallucinate stuff, it should actually be called confabulation when it's done by a language model. and they just make stuff up.
Now, psychologists don't say this so much because psychologists know that people just make stuff up.
Anybody who's studied memory going back to Bartlett in the 1930s,
knows that people are actually just like these large language models. They just invent stuff and for us, there's no hard line
between a true memory and a false memory.
If something happened recently and it sort of fits in with the things you understand, you'll probably remember
it roughly correctly. If something happened a long time ago, or it's weird, you'll remember it wrong, and often you'll be very confident
that you remembered it right, and you're just wrong. It's hard to show that. But one case where you can show it is John Dean's memory.
So John Dean testified at Watergate under oath. And retrospectively it's clear that he was trying to tell the truth.
But a lot of what he said was just plain wrong. He would confuse who was in which meeting,
he would attribute statements to other people who made that statement. And actually, it wasn't quite that statement.
He got meetings just completely confused, but he got the gist of what was going on in the White House right.
As you could see from the recordings. And because he didn't know the recordings, you could get a good experiment this way.

Ulric Neisser has a wonderful article talking about John Dean's memory, and he's just like a chat bot, he just make stuff up.
But it's plausible. So it's stuff that sounds good to him is what he produces.
They can also do reasoning. So I've got a friend in Toronto who is a symbolic AI guy, but very honest, so he's very confused by the fact these things work at all. and he suggested a problem to me.
I made the problem a bit harder and I gave this to GPT4 before it could look on the web.
So when it was just a bunch of weights frozen in 2021, all the knowledge is in the strength of the interactions between features.
So the rooms in my house are painted blue or white or yellow, yellow paint fades to white within a year. In two years time i want them all to be white.
What should I do and why? And Hector thought it wouldn't be able to do this.
And here's what you GPT4 said. It completely nailed it.
First of all, it started by saying assuming blue paint doesn't fade to white because after i told you yellow paint fades to white, well, maybe blue paint does too.
So assuming it doesn't, the white rooms you don't need to paint, the yellow rooms you don't need to paint because they're going to fade to white within a year.
And you need to paint the blue rooms white. One time when I tried it, it said, you need to paint the blue rooms yellow
because it realised that will fade to white. That's more of a mathematician's solution of reducing to a previous problem.
So, having claimed that these things really do understand, I want to now talk about some of the risks.
So, there are many risks from powerful AI. There's fake images, voices and video which are going to be used in the next election. There's many elections this year and they're going to help to undermine democracy. I'm very worried about that. The big companies are doing something about it, but maybe not enough.
There's the possibility of massive job losses. We don't really know about that. I mean, the past technologies often created jobs, but this stuff,
well, we used to be stronger, we used to be the strongest things around apart from animals.
And when we got the industrial revolution, we had machines that were much stronger. Manual labor jobs disappeared.
So the equivalent of manual labor jobs are going to disappear in the intellectual realm, and we get things that are much smarter than us.
So I think there's going to be a lot of unemployment. My friend Jen disagrees.
One has to distinguish two kinds of unemployment two, two kinds of job loss.
There'll be jobs where you can expand
the amount of work that gets done indefinitely. Like in health care. Everybody would love to have their own private doctors talking to them all the time.
So they get a slight itch here and the doctor says, no, that's not cancer. So there's room for huge expansion of how much gets done in medicine. So there won't be job loss there. But in other things, maybe there will be significant job loss.
There's going to be massive surveillance that's already happening in China. There's going to be lethal autonomous weapons
which are going to be very nasty, and they're really going to be autonomous. The Americans very clearly have already decided,
they say people will be in charge, but when you ask them what that means is it doesn't mean people will be in the loop that makes the decision to kill.

And as far as I know, the Americans intend to have half of their soldiers be robots by 2030.
Now, I do not know for sure that this is true. I asked Chuck Schumer's National Intelligence Advisor, and he said, well if there's anybody in the room who would know it would be me.
So, I took that to be the American way of saying, You might think that, but I couldn't possibly comment.
There's going to be cybercrime and deliberate pandemics.
I'm very pleased that in England, although they haven't done much towards regulation, they have set aside some money
so that they can experiment with open source models and see how easy it is to make them commit cyber crime.
That's going to be very important. There's going to be discrimination and bias. I don't think those are as important as the other threats.
But then I'm an old white male. Discrimination and bias I think are easier to handle than the other things.
If your goal is not to be unbiased. That your goal is to be less biased than the system you replace.
And the reason is if you freeze the weights of analysis, you can measure its bias and you can't do that with people.
They will change their behavior, once you start examining it. So I think discrimination bias of the ones where we can do quite a lot to fix them.
But the threat I'm really worried about and the thing I talked about after I left Google is the long term existential threat.
That is the threat that these things could wipe out humanity. And people were saying this is just science fiction.
Well, I don't think it is science fiction. I mean, there's lots of science fiction about it, but I don't think it's science fiction anymore.
Other people are saying the big companies are saying things like that to distract from all the other bad things.
And that was one of the reasons I had to leave Google before I could say this. So I couldn't be accused of being a Google stooge.
Although I must admit I still have some Google shares.
There's several ways in which they could wipe us out. So a superintelligence will be used by bad actors like Putin, Xi or Trump, and they'll want to use it for manipulating electorates and waging wars.
And they will make it do very bad things and they may may go too far and it may take over.
The thing that probably worries me most, is that if you want an intelligent agent that can get stuff done,
you need to give it the ability to create sub goals. So if you want to go to the states, you have a sub,
goal of getting to the airport and you can focus on that sub goal and not worry about everything else for a while.
So superintelligences will be much more effective if they're allowed to create sub goals.
And once they are allowed to do that, they'll very quickly realise there's an almost universal sub goal
which helps with almost everything. Which is get more control.

So I talked to a Vice President of the European Union about whether these things these things, will want to get control so that they could do things
better, the things we wanted, so they can do it better. Her reaction was, well why wouldn't they? We've made such a mess of it.
So she took that for granted. So they're going to have the sub go to getting more power
so they're more effective at achieving things that are beneficial for us and they'll find it easier to get more power
because they'll be able to manipulate people. So Trump, for example, could invade the Capital without ever going there himself.
Just by talking, he could invade the capital. And these superintelligences as long as they can talk to people
when they're much smarter than us, they'll be able to persuade us to do all sorts of things. And so I don't think there's any hope of a big switch that turns them off.
Whoever is going to turn that switch off will be convinced by the superintelligence. That's a very bad idea.
Then another thing that worries many people is what happens if superintelligences compete with each other?
You'll have evolution. The one that can grab the most resources will become the smartest.
As soon as they get any sense of self-preservation, then you'll get evolution occurring.
The ones with more sense of self-preservation will win and the more aggressive ones will win. And then you get all the problems that jumped up
Chimpanzees like us have. Which is we evolved in small tribes and we have lots of aggression and competition with other tribes.
And I want to finish by talking a bit about an epiphany I had at the beginning of 2023.
I had always thought that we were a long, long way away from superintelligence.
I used to tell people 50 to 100 years, maybe 3o to 100 years. It's a long way away. We don't need to worry about it now.
And I also thought that making our models more like the brain would make them better. I thought the brain was a whole lot better than the AI we had,
and if we could make AI a bit more like the brain, for example, by having three timescales,
most of the models we have at present have just two timescales. One for the changing of the weights, which is slow
and one for the words coming in, which is fast, changing the neural activities. So the changes in neural activities and changing in weights, the brain has more timescales than that. The brain has rapid changes in weight that quickly decayed away. And that's probably how it does a lot of short term memory.
And we don't have that in our models for technical reasons to do with being able to do matrix matrix multiplies.
I still believe that if once we got that into our models they'd get better, but
because of what I was doing for the two years previous to that, I suddenly came to believe that maybe the things we've got now,
the digital models, we've got now, are already very close to as good as brains and will get to be much better than brains.
Now I'm going to explain why I believe that. So digital computation is great.
You can run the same program on different computers, different piece of hardware or the same neural net on different pieces of hardware.

All you have to do is save the weights, and that means it's immortal once you've got some weights that are immortal.
Because if the hardware dies, as long as you've got the weights, you can make more hardware and run the same neural net.
But to do that, we run transistors at very high power, so they behave digitally and we have to have hardware that does exactly what you tell it to. That was great when we were instructing computers by telling them exactly how to do things, but we've now got another way of making computers do things.
And so now we have the possibility of using all the very rich analogue properties of hardware to get computations done at far lower energy.
So these big language models, when the training use like megawatts and we use 30 watts.
So because we know how to train things, maybe we could use analogue hardware and every piece of hardware is a bit different, but we train it to make use of its peculiar properties, so that it does what we want.
So it gets the right output for the input. And if we do that, then we can abandon the idea
that hardware and software have to be separate. We can have weights that only work in that bit of hardware
and then we can be much more energy efficient. So I started thinking
about what I call mortal computation, where you've abandoned that distinction between hardware and software using very low power analogue computation.
You can parallelize over trillions of weights that are stored as conductances.
And what's more, the hardware doesn't need to be nearly so reliable. You don't need to have hardware that at the level of the instructions would always do what you tell it to.
You can have goopy hardware that you grow and then you just learn to make it do the right thing.
So you should be able to use hardware much more cheaply, maybe even do some genetic engineering on neurons
to make it out of recycled neurons. I want to give you one example of how this is much more efficient.
So the thing you're doing in neural networks all the time is taking a vector of neural activities, and multiplying it by a matrix of weights, to get the vector
of neural activities in the next lane, at least get the inputs to the next lane. And so a vector matrix multiplies the thing you need to make efficient.
So the way we do it in the digital computer, is we have these transistors that are driven a very high power
to represent bits in say, a 32 bit number and then to multiply two 32 bit numbers, you need to perform.
I never did any computer science courses, but I think you need to perform about 1000 1 bit digital operations. It's about the square of the bitary.
If you want to do it fast. So you do lots of these digital operations.
There's a much simpler way to do it, which is you make a neural activity, be a voltage, you make a weight to be a conductance and a voltage times
a conductance is a charge, per unit time and charges just add themselves up.
So you can do your vector matrix multiply just by putting some voltages through some conductances.
And what comes into each neuron in the next layer will be the product of this vector with those weights.

That's great. It's hugely more energy efficient. You can buy chips to do that already, but every time you do
it'll be just slightly different. Also, it's hard to do nonlinear things like this.
So the several big problems with mortal computation, one is
that it's hard to use back propagation because if you're making use of the quirky analogue properties of a particular piece of hardware,
you can assume the hardware doesn't know its own properties. And so it's now hard to use the back propagation.
It's much easier to use reinforcement algorithms that tinker with weights to see if it helps. But they're very inefficient. For small networks.
We have come up with methods that are about as efficient as back propagation, a little bit worse.
But these methods don't yet scale up, and I don't know if they ever will Back propagation in a sense, is just the right thing to do.
And for big, deep networks, I'm not sure we're ever going to get things that work as well as back propagation.
So maybe the learning algorithm in these analogue systems isn't going to be as good as the one we have for things like large language models.
Another reason for believing that is, a large language model has say a trillion weights, you have 100 trillion weights.
Even if you only use 10% of them for knowledge, that's ten trillion weights. But the large language model in its trillion weights
knows thousands of times more than you do. So it's got much, much more knowledge.
And that's partly because it's seen much, much more data. But it might be because it has a much better learning algorithm.
We're not optimised for that. We're not optimised for packing lots of experience into a few connections where a trillion is a few.
We are optimized for having not many experiences.
You only live for about billion seconds. That's assuming you don't learn anything after you're 30, which is pretty much true.
So you live for about billion seconds and you've got 100 trillion connections,
so you've got crazily more parameters and you have experiences. So our brains optimise from making the best use of
not very many experiences. Another big problem with mortal computation is that if the software is inseparable from the hardware, once a system is learned or if the hardware dies, you lose,
all the knowledge, it's mortal in that sense. And so how do you get that knowledge into another mortal system?
Well, you get the old one to give a lecture and the new ones to figure out how to change the weights in their brains.
So they would have said that. That's called distillation. You try and get a student model to mimic
the output of a teacher model, and that works. But it's not that efficient.
Some of you may have noticed that universities just aren't that efficient. It's very hard to get the knowledge from the Professor into the student.
So this distillation method, a sentence, for example, has a few hundred bits of information, and even
if you learn optimally you can convey more than a few hundred bits. But if you take these big digital models,

then, if you look at a bunch of agents that all have exactly
the same neural netting with exactly the same weights and they're digital, so they use those weights in exactly the same way and these thousand different agents all go off
and look at different bits of the Internet and learn stuff. And now you want each of them to know what the other one's learned.
You can achieve that by averaging the gradients, so averaging the weights so you can get massive communication of what one agent learned to all the other agents.
So when you share the weight, so you share the gradients, you're communicating a trillion numbers, not just a few hundred bits, but a trillion real numbers.
And so they're fantastically much better at communicating, and that's what they have over us.
They're just much, much better at communicating between multiple copies of the same model.
And that's why GPT4 knows so much more than a human, it wasn't one model that did it.
It was a whole bunch of copies of the same model running on different hardware.
So my conclusion, which I don't really like,
is that digital computation requires a lot of energy, and so it would never evolve.
We have to evolve making use of the quirks of the hardware to be very low energy.
But once you've got it, it's very easy for agents to share
and GBT4 has thousands of times more knowledge in about 2% of the weights.
So that's quite depressing. Biological computation is great for evolving
because it requires very little energy, but my conclusion is
the digital computation is just better. And so I think it's fairly clear
that maybe in the next 20 years, I'd say with a probability of .5, in the next 20 years, it will get smarter than us
and very probably in the next hundred years it will be much smarter than us. And so we need to think about
how to deal with that. And there are very few examples of more intelligent
things being controlled by less intelligent things. And one good example is a mother being controlled by baby.
Evolution has gone to a lot of work to make that happen so that the baby survive, is very important for the baby to be able to control the mother.
But there aren't many other examples. Some people think that we can make these things be benevolent,
but if they get into a competition with each other, I think they'll start behaving like chimpanzees.
And I'm not convinced you can keep them benevolent. If they get very smart and they get any notion of self-preservation
they may decide they're more important than us. So I finish the lecture in record time.
I think.